Escuela Superior
de Ingeniería y Tecnología
Universidad de La Laguna



Informe de Prácticas: Vistas y Disparadores
Administración y Diseño de Bases de Datos

Integrantes:
Diego Vegas alu0101651361
César Navarro alu0101621302

2. Identificación de Objetos

```sql
SELECT table_name
FROM information_schema.tables
WHERE table_schema = 'public'
  AND table_type = 'BASE TABLE';
```

Data Output | Messages | Notifications

	table_name name
1	actor
2	store
3	address
4	category
5	city
6	country
7	customer
8	film_actor

```sql
SELECT table_name
FROM information_schema.views
WHERE table_schema = 'public';

```

Data Output Messages Notifications

SQL

	table_name name

```sql
SELECT sequence_name
FROM information_schema.sequences
WHERE sequence_schema = 'public';

```

Data Output Messages Notifications

SQL

	sequence_name name
1	customer_customer_id_...
2	actor_actor_id_seq
3	address_address_id_seq
4	category_category_id_seq
5	city_city_id_seq
6	country_country_id_seq
7	film_film_id_seq
8	inventory_inventory_id_s...

3. Tablas Principales y Relaciones

film almacena títulos y características; *actor* contiene intérpretes; *film_actor* vincula películas y actores; *category* y *film_category* organizan géneros; *inventory* representa ejemplares por tienda; *rental* registra alquileres; *payment* contiene importes; *customer* identifica clientes; *staff* y *store* describen personal y establecimientos. *address, city, country* y *language* aportan localización e idioma. Las claves primarias identifican filas y las foráneas conectan las tablas.

4. Consultas SQL

```sql
SELECT
    c.name AS categoria,
    SUM(p.amount) AS ventas_totales
FROM category c
JOIN film_category fc
    ON c.category_id = fc.category_id
JOIN inventory i
    ON fc.film_id = i.film_id
JOIN rental r
    ON i.inventory_id = r.inventory_id
JOIN payment p
    ON r.rental_id = p.rental_id
GROUP BY c.category_id, c.name
ORDER BY ventas_totales DESC;
```

Data Output | Messages | Notifications

	categoria character varying (25)	ventas_totales numeric
1	Sports	4892.19
2	Sci-Fi	4336.01
3	Animation	4245.31
4	Drama	4118.46
5	Comedy	4002.48
6	New	3966.38
7	Action	3951.84
8	Foreign	3934.47

```sql
SELECT
    s.store_id,
    ci.city || ', ' || co.country AS ubicacion,
    m.first_name || ' ' || m.last_name AS encargado,
    SUM(p.amount) AS ventas_totales
FROM store s
JOIN address ad ON s.address_id = ad.address_id
JOIN city ci ON ad.city_id = ci.city_id
JOIN country co ON ci.country_id = co.country_id
JOIN staff m ON s.manager_staff_id = m.staff_id
JOIN staff e ON e.store_id = s.store_id
JOIN payment p ON p.staff_id = e.staff_id
GROUP BY
    s.store_id, ci.city, co.country,
    m.first_name, m.last_name
ORDER BY ventas_totales DESC;
```

Data Output | Messages | Notifications

SQL

	store_id [PK] integer	ubicacion text	encargado text	ventas_totales numeric
1	2	Woodridge, Austra...	Jon Stephe...	31059.92
2	1	Lethbridge, Canada	Mike Hillyer	30252.12

```sql
SELECT
    f.film_id,
    f.title,
    f.description,
    c.name AS categoria,
    f.rental_rate AS precio,
    f.length AS duracion,
    f.rating AS clasificacion,
    STRING_AGG(
        a.first_name || ' ' || a.last_name,
        ', ' ORDER BY a.last_name, a.first_name
    ) AS actores
FROM film f
JOIN film_category fc ON f.film_id = fc.film_id
JOIN category c ON fc.category_id = c.category_id
JOIN film_actor fa ON f.film_id = fa.film_id
JOIN actor a ON fa.actor_id = a.actor_id
GROUP BY f.film_id, c.name
```

Data Output Messages Notifications

Showing rows: 1 to 997 Page No: 1 of 1

	film_id integer	title character varying (255)	description text	categoria character varying (25)	precio numeric (4,2)	duracion smallint
1	1	Academy Dinosaur	A Epic Drama of a Feminist And a Mad Scientist who must Battle a Teacher in The Canadian Rockies	Documentary	0.99	86
2	2	Ace Goldfinger	A Astounding Epistle of a Database Administrator And a Explorer who must Find a Car in Ancient China	Horror	4.99	48
3	3	Adaptation Holes	A Astounding Reflection of a Lumberjack And a Car who must Sink a Lumberjack in A Baloon Factory	Documentary	2.99	50
4	4	Affair Prejudice	A Fanciful Documentary of a Frisbee And a Lumberjack who must Chase a Monkey in A Shark Tank	Horror	2.99	117
5	5	African Egg	A Fast-Paced Documentary of a Pastry Chef And a Dentist who must Pursue a Forensic Psychologist in The Gulf of Mexico	Family	2.99	130
6	6	Agent Truman	A Intrepid Panorama of a Robot And a Boy who must Escape a Sumo Wrestler in Ancient China	Foreign	2.99	169
7	7	Airplane Sierra	A Touching Saga of a Hunter And a Butler who must Discover a Butler in A Jet Boat	Comedy	4.99	62

```sql
SELECT
    a.actor_id,
    a.first_name,
    a.last_name,
    STRING_AGG(DISTINCT c.name, ':') AS categorias,
    STRING_AGG(DISTINCT f.title, ':') AS peliculas
FROM actor a
JOIN film_actor fa ON a.actor_id = fa.actor_id
JOIN film f ON fa.film_id = f.film_id
JOIN film_category fc ON f.film_id = fc.film_id
JOIN category c ON fc.category_id = c.category_id
GROUP BY a.actor_id, a.first_name, a.last_name
ORDER BY a.actor_id;
```

Data Output Messages Notifications

Showing rows: 1 to 200 Page No: 1 of 1

	actor_id [PK] integer	first_name character varying (45)	last_name character varying (45)	categorias text	peliculas text
1	1	Penelope	Guiness	Animation:Children:Classics:Comedy:Documentary:Family:Foreign:Games:Horror:Music:New:Sci-Fi:Sports	Academy Dinosaur:Anaconda Confessior
2	2	Nick	Wahlberg	Action:Animation:Children:Classics:Comedy:Documentary:Drama:Family:Foreign:Games:Music:New:Sci-Fi:Travel	Adaptation Holes:Apache Divine:Baby Ha
3	3	Ed	Chase	Action:Classics:Documentary:Drama:Foreign:Music:New:Sci-Fi:Sports:Travel	Alone Trip:Army Flintstones:Artist Coldbl
4	4	Jennifer	Davis	Action:Animation:Comedy:Documentary:Drama:Family:Horror:Music:New:Sci-Fi:Sports:Travel	Anaconda Confessions:Angels Life:Baref
5	5	Johnny	Lollobrigida	Action:Animation:Children:Comedy:Documentary:Drama:Games:Horror:Music:New:Sci-Fi:Sports:Travel	Amadeus Holy:Banger Pinocchio:Bonnie
6	6	Bette	Nicholson	Action:Animation:Children:Classics:Documentary:Family:Foreign:New:Sci-Fi:Sports:Travel	Antitrust Tomatoes:Bang Kwai:Beast Hun
7	7	Grace	Mostel	Action:Animation:Children:Classics:Drama:Foreign:Games:Horror:New:Sci-Fi:Sports	Angels Life:Anonymous Human:Arachnoy

Successfully run. Total query runtime: 76 msec. 200 rows affected.

5. Creación de Vistas

```sql
CREATE VIEW view_ventas_categoria AS

SELECT
    c.name AS categoria,
    SUM(p.amount) AS ventas_totales
FROM category c
JOIN film_category fc
    ON c.category_id = fc.category_id
JOIN inventory i
    ON fc.film_id = i.film_id
JOIN rental r
    ON i.inventory_id = r.inventory_id
JOIN payment p
    ON r.rental_id = p.rental_id
GROUP BY c.category_id, c.name
ORDER BY ventas_totales DESC;
```

Data Output | Messages | Notifications

```
CREATE VIEW

Query returned successfully in 65 msec.
```

```sql
CREATE VIEW view_ventas_tienda AS
SELECT
    s.store_id,
    ci.city || ', ' || co.country AS ubicacion,
    m.first_name || ' ' || m.last_name AS encargado,
    SUM(p.amount) AS ventas_totales
FROM store s
JOIN address ad ON s.address_id = ad.address_id
JOIN city ci ON ad.city_id = ci.city_id
JOIN country co ON ci.country_id = co.country_id
JOIN staff m ON s.manager_staff_id = m.staff_id
JOIN staff e ON e.store_id = s.store_id
JOIN payment p ON p.staff_id = e.staff_id
GROUP BY
    s.store_id, ci.city, co.country,
    m.first_name, m.last_name
ORDER BY ventas_totales DESC;

```

Data Output | Messages | Notifications

```
CREATE VIEW

Query returned successfully in 42 msec.
```

```sql
CREATE VIEW view_peliculas_actores  AS SELECT
    f.film_id,
    f.title,
    f.description,
    c.name AS categoria,
    f.rental_rate AS precio,
    f.length AS duracion,
    f.rating AS clasificacion,
    STRING_AGG(
        a.first_name || ' ' || a.last_name,
        ', ' ORDER BY a.last_name, a.first_name
    ) AS actores
FROM film f
JOIN film_category fc ON f.film_id = fc.film_id
JOIN category c ON fc.category_id = c.category_id
JOIN film_actor fa ON f.film_id = fa.film_id
JOIN actor a ON fa.actor_id = a.actor_id
GROUP BY f.film_id, c.name
ORDER BY f.film_id;

```

Data Output | Messages | Notifications

```
CREATE VIEW

Query returned successfully in 40 msec.
```

```sql
CREATE VIEW view_ventas_peliculas AS
SELECT
    a.actor_id,
    a.first_name,
    a.last_name,
    STRING_AGG(DISTINCT c.name, ':') AS categorias,
    STRING_AGG(DISTINCT f.title, ':') AS peliculas
FROM actor a
JOIN film_actor fa ON a.actor_id = fa.actor_id
JOIN film f ON fa.film_id = f.film_id
JOIN film_category fc ON f.film_id = fc.film_id
JOIN category c ON fc.category_id = c.category_id
GROUP BY a.actor_id, a.first_name, a.last_name
ORDER BY a.actor_id;
```

Data Output | Messages | Notifications

```
CREATE VIEW

Query returned successfully in 43 msec.
```

6. Restricciones CHECK

Se añadieron condiciones de validación para impedir duraciones de película y alquiler no positivas, tarifas y costes de sustitución negativos, pagos negativos y devoluciones anteriores a la fecha de alquiler. En la captura se observa la ejecución correcta de las sentencias *ALTER TABLE*. Una devolución *NULL* sigue siendo válida para representar un alquiler pendiente de retorno. El dominio *year* ya incorpora una validación para el año de estreno.

```sql
ALTER TABLE film
ADD CONSTRAINT check_film_length
CHECK (length > 0);

ALTER TABLE film
ADD CONSTRAINT check_rental_duration
CHECK (rental_duration > 0);

ALTER TABLE film
ADD CONSTRAINT check_rental_rate
CHECK (rental_rate >= 0);

ALTER TABLE film
ADD CONSTRAINT check_replacement_cost
CHECK (replacement_cost >= 0);

ALTER TABLE payment
ADD CONSTRAINT check_payment_amount
CHECK (amount >= 0);

ALTER TABLE rental
ADD CONSTRAINT check_return_date
CHECK (return_date >= rental_date);

```

Data Output | Messages | Notifications

```
ALTER TABLE

Query returned successfully in 45 msec.
```

7. Disparador last_updated

El disparador *last_updated BEFORE UPDATE ON customer FOR EACH ROW EXECUTE PROCEDURE last_updated()* ejecuta una función antes de modificar cada fila de la tabla *customer*. La función asigna *CURRENT_TIMESTAMP* a *NEW.last_update* y devuelve *NEW*, manteniendo automáticamente la fecha de modificación. La tabla *actor* utiliza una solución similar.

8. Registro de Inserciones

Se creó la tabla *film_insert_log* con los campos *id, film_id* y *fecha*; una función PL/pgSQL toma *NEW.film_id* y guarda la marca temporal actual. Un trigger *AFTER INSERT ON film* la invoca por cada fila insertada. La captura aportada muestra el registro del *film_id* 1001.

```sql
-- 1. Crear la tabla de registro
CREATE TABLE film_insert_log (
    id SERIAL PRIMARY KEY,
    film_id INTEGER NOT NULL,
    fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 2. Crear la función del trigger
CREATE OR REPLACE FUNCTION registrar_film_insert()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO film_insert_log (film_id, fecha)
    VALUES (NEW.film_id, CURRENT_TIMESTAMP);

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

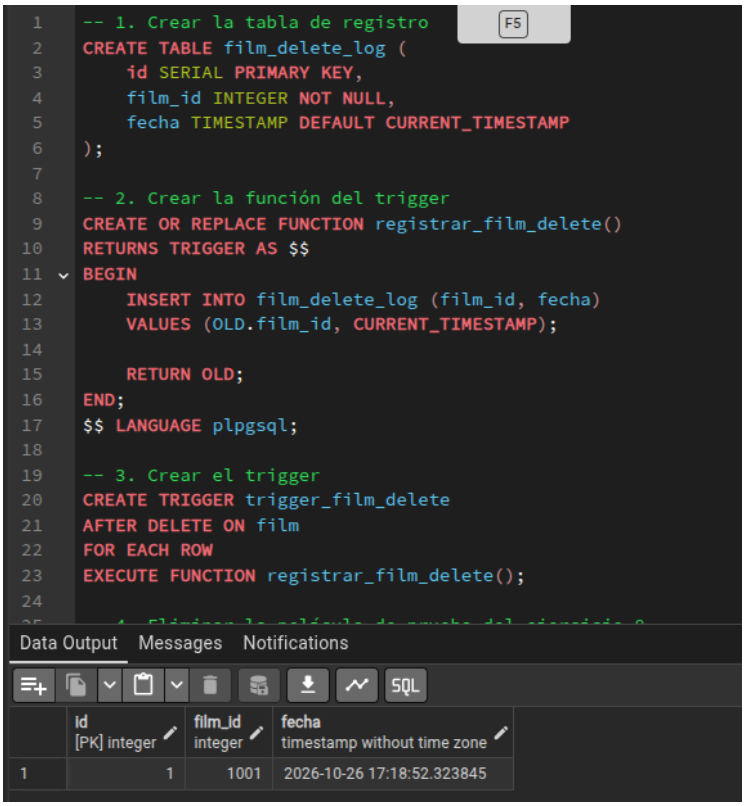
-- 3. Crear el trigger
CREATE TRIGGER trigger_film_insert
AFTER INSERT ON film
FOR EACH ROW
EXECUTE FUNCTION registrar_film_insert();
```

Data Output | Messages | Notifications

	id [PK] integer	film_id integer	fecha timestamp without time zone
1	1	1001	2026-10-26 17:18:01.773194

9. Registro de Eliminaciones

Se creó la tabla *film_delete_log* con los campos *id, film_id* y *fecha*; una función PL/pgSQL utiliza *OLD.film_id* y registra la fecha al eliminar una película. El trigger *AFTER DELETE ON film* ejecuta la función por cada fila borrada. La captura aporta la traza del *film_id* 1001 tras su eliminación



```sql
-- 1. Crear la tabla de registro
CREATE TABLE film_delete_log (
    id SERIAL PRIMARY KEY,
    film_id INTEGER NOT NULL,
    fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 2. Crear la función del trigger
CREATE OR REPLACE FUNCTION registrar_film_delete()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO film_delete_log (film_id, fecha)
    VALUES (OLD.film_id, CURRENT_TIMESTAMP);

    RETURN OLD;
END;
$$ LANGUAGE plpgsql;

-- 3. Crear el trigger
CREATE TRIGGER trigger_film_delete
AFTER DELETE ON film
FOR EACH ROW
EXECUTE FUNCTION registrar_film_delete();
```

Data Output | Messages | Notifications

	id [PK] integer	film_id integer	fecha timestamp without time zone
1	1	1001	2026-10-26 17:18:52.323845

10. Secuencias

Las secuencias generan valores numéricos secuenciales, principalmente para asignación de claves primarias. En la tabla *film*, la secuencia *film_film_id_seq* permite asignar *film_id* automáticamente mediante la función *nextval()*. Resultan fundamentales para inserciones concurrentes y evitan el cálculo manual de nuevos identificadores, si bien no garantizan una numeración estrictamente correlativa sin saltos.